



AI Security Scanner for Python



Shivam Arora

```
...python
def authenticate(username, password):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users WHERE username = ? AND password = ?", (username, password))
    return cursor.fetchone()
...

**2. Vulnerability Type:** Weak Cryptography / Insecure Password Hashing
* **Why it is vulnerable:** MD5 is a cryptographically broken hash function and is not suitable for securely storing passwords.
* **Impact:** Attackers can easily crack hashed passwords using rainbow tables or brute-force attacks due to the algorithm's weakness and lack of salting.
* **Secure code fix:** Use a strong, modern, and salted password hashing algorithm like `bcrypt` (via `passlib`) or `argon2`.
...python
import bcrypt # pip install bcrypt

def hash_password(password):
    salt = bcrypt.gensalt()
    hashed_password = bcrypt.hashpw(password.encode('utf-8'), salt)
    return hashed_password.decode('utf-8') # Store as string

# For verification: bcrypt.checkpw(password.encode('utf-8'), stored_hash.encode('utf-8'))
...

**3. Vulnerability Type:** Command Injection
* **Why it is vulnerable:** User-controlled input ('user_input') is directly embedded into a shell command executed via `os.system()`, allowing injection of arbitrary commands.
* **Impact:** An attacker can execute arbitrary operating system commands with the privileges of the application, leading to full system compromise.
* **Secure code fix:** Avoid `os.system()` with user input; prefer safer alternatives like `subprocess.run()` with `shell=False` and input passed as arguments.
...python
import subprocess

def process_input(user_input):
    # Safely pass user_input as an argument, preventing shell interpretation
    subprocess.run(["echo", user_input], check=True)
...

**4. Vulnerability Type:** Hardcoded Secrets
* **Why it is vulnerable:** Sensitive credentials ('DB_PASSWORD', 'API_SECRET') are embedded directly within the source code, making them easily discoverable if the code is accessed.
* **Impact:** Compromise of the code repository or binary could expose sensitive credentials, leading to unauthorized access to databases or APIs.
* **Secure code fix:** Store credentials in environment variables, a secure secrets manager, or a configuration file that is not committed to version control.
...python
import os

DB_PASSWORD = os.getenv("DB_PASSWORD", "default_safe_password") # Provide a safe default or make it mandatory
API_SECRET = os.getenv("API_SECRET", "default_safe_secret")
...

**5. Vulnerability Type:** Information Leakage (Sensitive Data Exposure)
* **Why it is vulnerable:** The `connect_to_database` function explicitly returns a string containing the hardcoded database password.
* **Impact:** If this return value is ever logged, displayed, or accidentally exposed, it could lead to the database password being revealed to unauthorized parties.
* **Secure code fix:** Do not return or log sensitive information like passwords directly; handle connection objects or success/failure indicators.
...python
def connect_to_database():
    # Example: Actual connection would return a connection object
    # conn = sqlite3.connect('users.db', password=DB_PASSWORD) # (omitted as per requested removal)
    ...
```



Introducing Today's Project!

This project aims to build a simple Python-based security scanner using the Gemini API. The scanner helps developers identify security vulnerabilities in Python code automatically instead of checking the code manually. The user provides a Python file to the scanner, which reads the code and sends it to Gemini AI for analysis. Gemini examines the code and detects common security issues such as SQL injection, hardcoded secrets (passwords or API keys), and weak cryptographic practices. After analyzing the code, Gemini returns a report describing the vulnerabilities and possible improvements. The scanner then displays the results in a clear and readable format in the command line. This project demonstrates how artificial intelligence can be used to improve software security and save development time.

Key tools and concepts

The key tools I used include in this is Gemini API, Python, Colorama Libraries. Key concepts I learnt include prompt structuring, code basics and terminal commands.

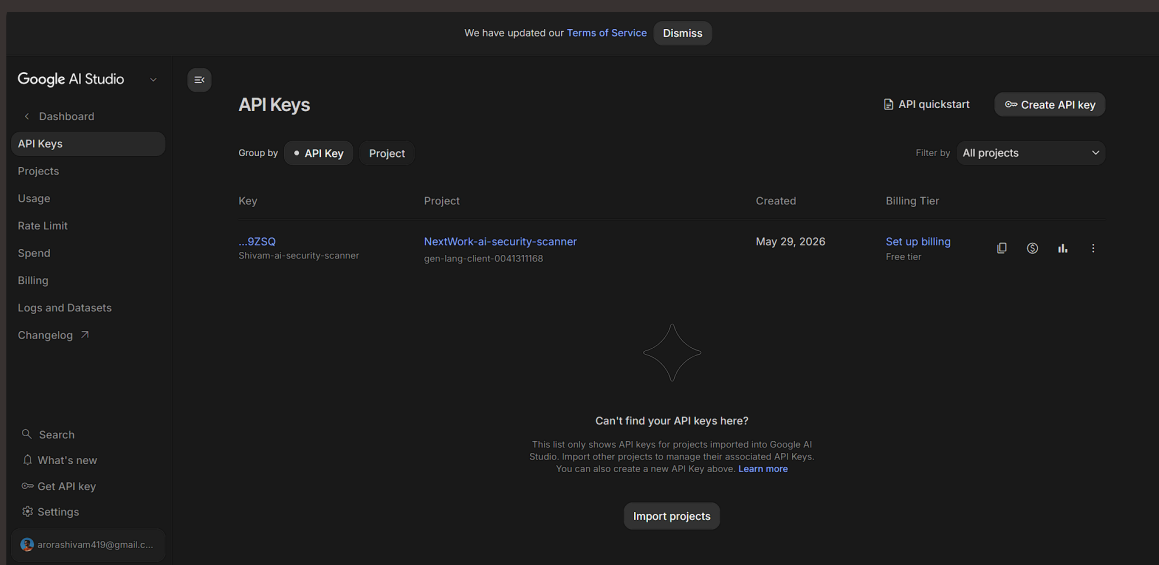
Challenges and wins

Today, I completed this project to learn how to integrate the Gemini API key into an application and use it for AI-powered security analysis. I also learned about different types of security vulnerabilities and how a security scanner can identify them. Through this project, I gained hands-on experience with API integration, environment variables, and vulnerability detection. Moving forward, I would like to learn how to implement more advanced features, support additional programming languages, improve vulnerability detection accuracy, and make the scanner more generalized and scalable for real-world use cases. This project helped me strengthen my understanding of both AI integration and cybersecurity concepts.



Generating the Gemini API Key

In this step, we generate a Gemini API Key from Google. The API key is a unique credential that enables our Python security scanner to communicate with the Gemini AI service. Since the scanner depends on Gemini to analyze Python code and detect security vulnerabilities, obtaining the API key is essential. Once generated, the key will be added to our application so it can send code to Gemini and receive security analysis results. This is the first setup step for the project and ensures that the scanner has access to Gemini's AI capabilities to identify potential security issues in code.





Setting Up the Python Environment

In this step, we are setting up the Python environment required for building the security scanner. First, we create a new project folder to organize all project files. Next, we install Python, which will be used to write and run the scanner application. Finally, we create a virtual environment to manage project dependencies separately from other Python projects on the system. A virtual environment helps avoid package conflicts and keeps the project organized. Completing these setup tasks ensures that the development environment is ready, allowing us to start coding and integrating the Gemini API into our security scanner.

```
PS C:\MCA\security-scanner> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\MCA\security-scanner\venv\Scripts\Activate.ps1)
(venv) PS C:\MCA\security-scanner> python
Python 3.14.5 (tags/v3.14.5:5607950, May 10 2026, 10:43:50) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
Ctrl click to launch VS Code Native REPL
>>> ^Z

(venv) PS C:\MCA\security-scanner> venv\Scripts\activate
>>
❖ (venv) PS C:\MCA\security-scanner> |
```

Installing the required packages

Now that I have my virtual environment set up, I need to install two Python packages (python-dotenv) for securely storing the Gemini API Key, and another package (google-gemini), which recognizes the Gemini calls that were written in Python. This is important for my project because these packages enable secure



Installing them ensures that all required dependencies are available, helping the project run smoothly, securely, and without configuration errors during development and testing.

```
(venv) PS C:\MCA\security-scanner> pip show google-genai
Name: google-genai
Version: 2.7.0
Summary: GenAI Python SDK
Home-page: https://github.com/googleapis/python-genai
Author:
Author-email: Google LLC <googleapis-packages@google.com>
License-Expression: Apache-2.0
Location: C:\MCA\security-scanner\venv\Lib\site-packages
Requires: anyio, distro, google-auth, httpx, pydantic, requests, sniffio, tenacity, typing-extensions, websockets
Required-by:
(venv) PS C:\MCA\security-scanner>
```



Connecting to the Gemini API

In this step, I'm connecting to Gemini!. I'm going to write my first bit of code for my scanner.py script and then test it. This is important because I need to connect to Gemini with my API key.

```
(venv) PS C:\MCA\security-scanner> python venv/scanner.py
Traceback (most recent call last):
  File "C:\MCA\security-scanner\venv\scanner.py", line 7, in <module>
    client = genai.Client(api_key=api_key)
  File "C:\MCA\security-scanner\venv\Lib\site-packages\google\genai\client.py", line 458, in __init__
    self._api_client = self._get_api_client(
                        ~~~~~^
                        vertexai=resolved_vertexai,
                        ~~~~~^
    ...<5 lines>...
                        http_options=http_options,
                        ~~~~~^
    )
    ^
  File "C:\MCA\security-scanner\venv\Lib\site-packages\google\genai\client.py", line 506, in _get_api_client
    return BaseApiClient(
           vertexai=vertexai,
    ...<4 lines>...
           http_options=http_options,
    )
  File "C:\MCA\security-scanner\venv\Lib\site-packages\google\genai\_api_client.py", line 783, in __init__
    raise ValueError(
    ...<3 lines>...
    )
ValueError: No API key was provided. Please pass a valid API key. Learn how to create an API key at
https://ai.google.dev/gemini-api/docs/api-key.
(venv) PS C:\MCA\security-scanner>
```

Securing the API key with a .env file

I got an error! My error was that exposing an API key directly in the source code can lead to security risks and unauthorized access. Having a .env file is important because it stores sensitive information securely outside the code, preventing accidental exposure when sharing code or uploading projects to GitHub. It also protects the API quota and helps follow secure development practices.



```
1. **Sunlight is White Light:** Sunlight, or "white light," is actually composed of all the colors of the rainbow, each with a different wavelength. Red light has the longest wavelength, and violet/blue light has the shortest wavelength.

2. **Earth's Atmosphere:** Our atmosphere is made up of tiny gas molecules, primarily nitrogen (about 78%) and oxygen (about 21%). These molecules are much smaller than the wavelengths of visible light.

3. **Rayleigh Scattering:** When sunlight enters the atmosphere, it collides with these tiny gas molecules. Because the molecules are so small relative to the wavelengths of light, they scatter light preferentially based on its wavelength.
   * **Shorter wavelengths (blue and violet light) are scattered much more efficiently** than longer wavelengths (red, orange, and yellow light). Blue light is scattered about 10 times more effectively than red light.

4. **Why We See Blue:**
   * As sunlight travels through the atmosphere, most of the blue and violet light gets scattered in **all directions** by these tiny molecules.
   * This scattered blue light reaches our eyes from every part of the sky, making it appear blue.
   * The longer wavelengths (red, yellow, orange) are scattered much less and tend to travel more directly from the sun to our eyes.

5. **Why Not Violet?*** You might wonder why the sky isn't violet, since violet light has an even shorter wavelength than blue and scatters even more. There are two main reasons:
   * The sun emits slightly less violet light than blue light.
   * Our eyes are more sensitive to blue light than to violet light.

**In summary:** The tiny molecules in Earth's atmosphere act like tiny prisms, scattering blue light more effectively than other colors, causing the sky to appear blue from our perspective.

**Bonus: Why are sunsets and sunrises red/orange?***
At sunrise and sunset, sunlight has to travel through a much greater amount of atmosphere to reach our eyes. By the time it gets to us, nearly all the blue and violet light has been scattered away, leaving mostly the longer-wavelength red, orange, and yellow light to pass directly through. This is
```

Verifying the connection

I verified the connection by running my script. Gemini responded with an answer that the sky is blue because of a very important reason, which confirmed that my API Key connection is working well.



Engineering the Security Prompt

In this step, we are teaching Gemini to act like a cybersecurity expert instead of a normal chatbot. We give it special instructions on how to check computer code for security problems, bugs, and weaknesses. Then, we test it to make sure it understands these instructions and can find security issues correctly. In simple words, we are training Gemini to become the "security inspector" of our project so it can help find and explain problems in code.

```
scanner.py > ...
1  import os
2  from dotenv import load_dotenv
3  import google.generativeai as genai
4  load_dotenv()
5  api_key = os.getenv("GOOGLE_API_KEY")
6
7  client = genai.Client(api_key=api_key)
8  security_prompt = """
9  You are a security expert. Analyze this code for vulnerabilities.
10
11  For each issue, provide:
12  1. Vulnerability type
13  2. Why it is vulnerable (1 sentence)
14  3. Impact (1 sentence)
15  4. Secure code fix
16
17  Be concise.
18
19  Code:
20  {code}
21  """
22  # Test with a simple prompt
23  try:
24      response = client.models.generate_content(
25          model='gemini-2.5-flash', contents=security_prompt
26      )
27      print(response)
28  except Exception as e:
29      print(f"❌ Connection failed: {e}")
30
```




Building the File Scanner

In this step, I'm adding file scanning so my scanner can read other files I send it. Then I am going to update my Gemini calls to send real code and verify it's ready. This is important because it's the core of my scanner.

```
scanner.py X .env
scanner.py 2 --
1 import os
2 import sys
3 from dotenv import load_dotenv
4 import google.generativeai as genai
5 load_dotenv()
6 api_key = os.getenv("GOOGLE_API_KEY")
7 client = genai.Client(api_key=api_key)
8 security_prompt = """
9 You are a security expert. Analyze this code for vulnerabilities.
10
11 For each issue, provide:
12 1. Vulnerability type
13 2. Why it is vulnerable (1 sentence)
14 3. Impact (1 sentence)
15 4. Secure code fix
16
17 Be concise.
18
19 Code:
20 {code}
21 """
22 # File path from command line: python scanner.py <file_path>
23 if len(sys.argv) < 2:
24     print("Usage: python scanner.py <file_path>")
25     sys.exit(1)
26
27 code_path = sys.argv[1]
28 with open(code_path, "r") as f:
29     code = f.read()
30 prompt = security_prompt.format(code=code)
31
32 # Test with a simple prompt
33 try:
34     response = client.models.generate_content(
35         model="gemini-2.5-flash", contents=prompt
36     )
37     print(response.text)
38 except Exception as e:
39     print(f"❌ Connection failed: {e}")
40
```



Running the Security Audit

In this step, I'm testing my scanner by creating a sample code file with a security vulnerability and running it through my scanner. I'll start with one vulnerability first because it makes it easier to check whether the scanner is working correctly. Once I confirm that the scanner can detect the first issue, I'll add more vulnerabilities and scan again to see how well it identifies multiple security problems. This helps me verify that my scanner is accurately analyzing code and reporting security risks.

```
(venv) PS C:\MCA\security-scanner> python scanner.py vulnerable.py
>>
Here's an analysis of the provided code for vulnerabilities:

---

### Issue 1: Hardcoded Credentials

1. **Vulnerability type:** Hardcoded Credentials / Sensitive Data Exposure
2. **Why it is vulnerable:** Sensitive information (database password and API secret) is directly embedded in the source code.
3. **Impact:** Anyone with access to the code repository or compiled application can easily extract these credentials, leading to full compromise of the database and API.
4. **Secure code fix:** Store credentials in environment variables, a secure configuration management system, or a dedicated secret management service (e.g., AWS Secrets Manager, HashiCorp Vault) and retrieve them at runtime.

---

### Issue 2: Sensitive Data Exposure in Return Value

1. **Vulnerability type:** Sensitive Data Exposure / Information Disclosure
2. **Why it is vulnerable:** The `connect_to_database` function directly embeds and returns the raw database password in its output string.
3. **Impact:** Any system logging the return value of this function, or displaying it to users, will expose the database password, enabling unauthorized access.
4. **Secure code fix:** Never include sensitive information like passwords directly in return values, log messages, or user-facing output; instead, establish a connection securely and return a connection object or handle the actual connection logic internally.
(venv) PS C:\MCA\security-scanner>
```

Analyzing the vulnerability report

My scanner detected the 4 vulnerabilities. The most surprising vulnerability for me was the "f" bomb. This shows that AI can help with security by identifying issues that might be beyond your control or that you don't know about.



Adding Color-Coded Severity Ratings

In this project extension, I'm adding the Colorama library to my script. This lets the scanner prioritize the vulnerabilities based on the color, so that high critical vulnerabilities are fixed first

```
---
SEVERITY: CRITICAL
TYPE: SQL Injection
DESCRIPTION: User-supplied input for username and password is directly concatenated into an SQL query without proper sanitization.
IMPACT: An attacker can bypass authentication, access, modify, or delete arbitrary data in the database.
FIX:
```python
cursor.execute("SELECT * FROM users WHERE username = ? AND password = ?", (username, password))
```
---
SEVERITY: HIGH
TYPE: Weak Cryptography
DESCRIPTION: The MD5 hash function is used for password hashing, which is cryptographically weak and vulnerable to collision and brute-force attacks.
IMPACT: Passwords stored using MD5 can be easily cracked, compromising user accounts if the database is breached.
FIX:
```python
import hashlib
def hash_password(password):
 return hashlib.sha256(password.encode()).hexdigest()
```
---
SEVERITY: CRITICAL
TYPE: OS Command Injection
DESCRIPTION: User-supplied input is directly inserted into an operating system command executed via `os.system()`, allowing arbitrary command execution.
IMPACT: An attacker can execute arbitrary commands on the host system, leading to full system compromise or data theft.
FIX:
```python
import shlex
os.system(f"echo {shlex.quote(user_input)}")
```
---
SEVERITY: HIGH
TYPE: Hardcoded Secrets
DESCRIPTION: Sensitive credentials (database password, API secret) are hardcoded directly into the source code.
IMPACT: If the source code is compromised, these secrets will be immediately available to attackers, leading to unauthorized access to databases or APIs.
FIX:
```python
import os
DB_PASSWORD = os.getenv("DB_PASSWORD")
API_SECRET = os.getenv("API_SECRET")
```
```

How the color system works

I added colour to my responses by using the colorama library and updating my prompt + my calls to the gemini to include the colours. So that the severity of the error could be defined easily with the help of this.



Wrapping Up

This project took me approximately 1 hour. The most challenging part was adding the coloroma part as well as the sys part in the Python script.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

